INTEGRATION GUIDE

Restaurant website and ERP integration

What was changed to connect the RestNext website to the ERP (demos3): the hero slider, category bar, deals, menu, orders and delivery areas. Each section lists the files that changed, before and after.

CONTENTS

1. Overview
2. How it fits together
3. Before and after
4. Hero slider
5. Category bar
6. Deals
7. Menu, sizes and modifiers
8. Orders and delivery areas
9. Files changed
10. Running and checking
11. Open items

1. Overview

The website used to show sample data: a fixed hero, fixed deals, fixed categories and a fake order tracker. After the integration:

- The menu, categories, popular items, deals and hero come from the ERP (the `restaurant/api` service reads the ERP database).
- Orders placed on the website appear in the ERP's Web Orders screen. Completing an order posts a Sales Invoice.
- Delivery charges depend on the area the customer picks, and each area's charge is set in the ERP.

- If the ERP API is down, the website falls back to its sample content, so the site still loads.

2. How it fits together

ERP (demos3)
Stock items, attributes, prices, deals, delivery areas, Web Orders screens. PHP on XAMPP.

Restaurant API
`htdocs/restaurant/api/index.`
Reads and writes the `rs_` tables in `demofinac` .

RestNext website
Next.js. Calls the API on the server (menu, deals, areas) and through a same-origin proxy in the browser.

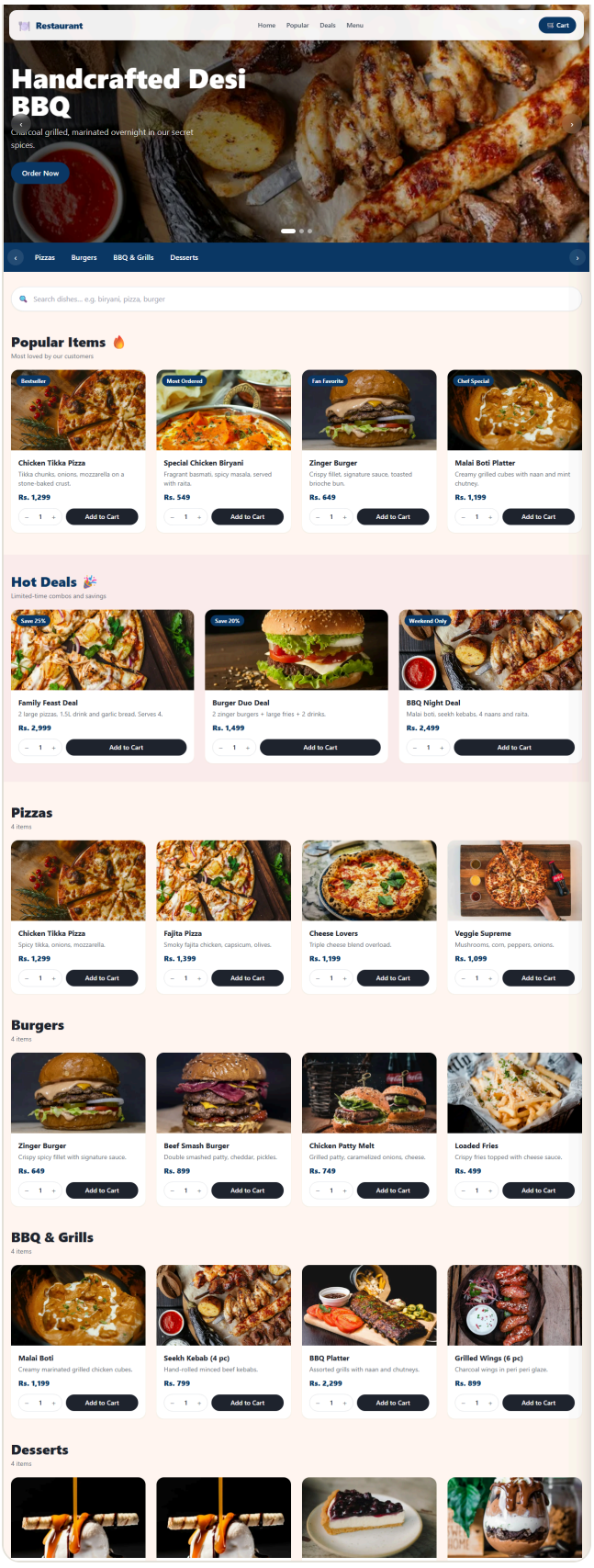
Routes the website uses: `/menu` , `/deals` , `/areas` , `/orders` , `/health` . The proxy allows only these, so nothing else is exposed.

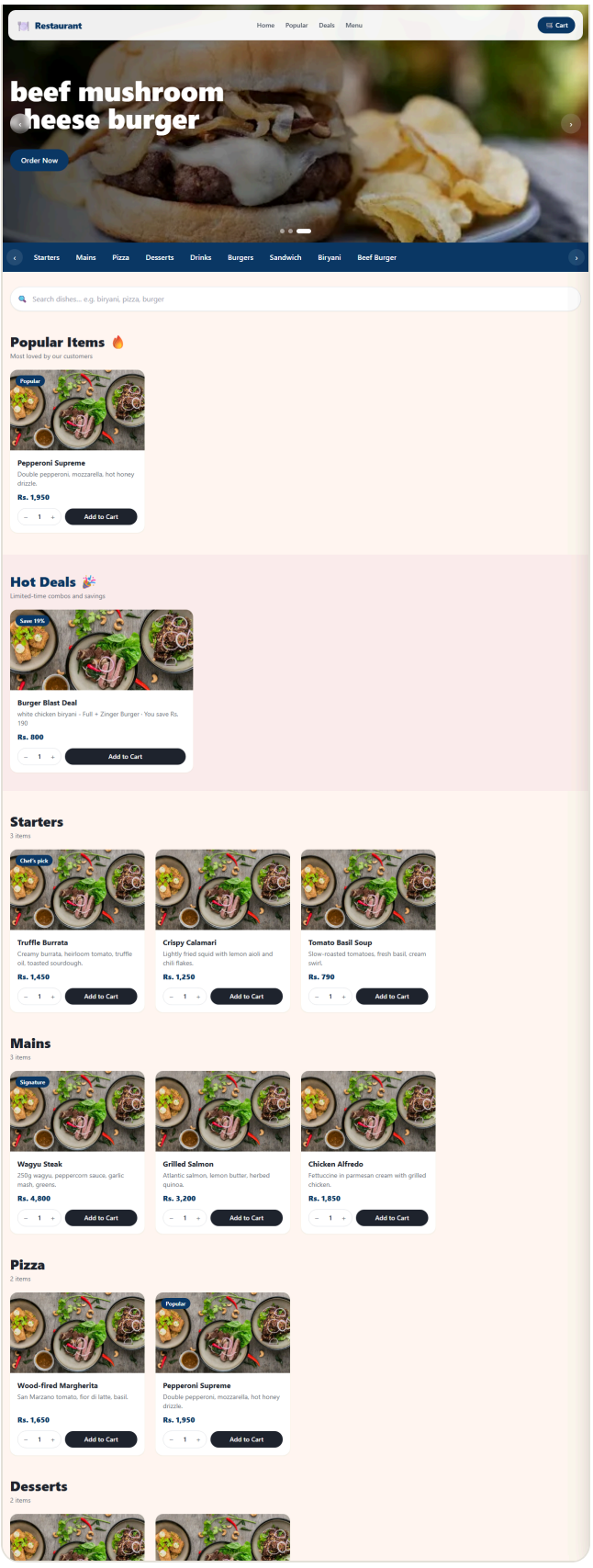
3. Before and after

Home page, full length. The "before" image is the sample content the site showed before the ERP was connected. The "after" image is the same page with live ERP data.

Before: sample data

After: live ERP data





Screenshots taken with Microsoft Edge (headless) at 1280px wide. The page is shown at full length.

4. Hero slider

The hero slider now shows only the dishes marked in the ERP. Nothing else appears there.

In the ERP

- **Stock → Item Report → item form → Restaurant Website:** new switch `Show on hero slider (homepage banner)`. It needs the dish to be on the website menu too.
- Saved by `saveHeroFromStockForm()` in `cls_RestaurantMenu.php`, which sets `rs_menu_items.show_in_hero`.

On the API and the website

- `GET /menu` sends `hero: true/false` on each dish.
- RestNext builds `heroSlides` from dishes with `hero` set and `available` true. Each slide uses the dish name, description and image. Button text: "Order Now".
- No flagged dish means no hero section. The sample hero is used only when the API is down.

> **Result:** Zinger Burger was flagged, so the hero shows only Zinger Burger. Clearing the flag hides the hero section.

5. Category bar

The categories come from the ERP dish categories. Each category becomes a button in a horizontal bar, and clicking it scrolls to that category's section.

Before

- Square photo cards with a title under each.
- The row scrolled horizontally with no visible scrollbar and no arrows, so the last categories could be missed.

After

- A full-width `bg-primary` bar with text links. Names are shown capitalised ("Beef Burger").
- Links are white by default. On hover, the background turns white and the text turns navy.

- Left and right arrows on desktop move the bar. On phones, swiping works.

Files: `restnext/components/CategorySlider.tsx`, `restnext/lib/data.ts` (categories are grouped from `/menu` by the dish's category name).

6. Deals

A deal is a fixed price for two or more items from the ERP stock. The website shows the deal and sells it as its items, so stock is still taken per item.

In the ERP

- **Restaurant → Deals tab:** create or edit a deal. Enter the name, description, deal price and "Show on website". Pick the items in the Items table by typing the stock item name (autocomplete). Each item needs one attribute at most.
- The save checks that there are at least 2 different items, and that each item is on the website menu.
- **Deals Report:** one row per deal with its orders, units sold and revenue. The `eye` icon opens the deal view with its items and recent orders.
- Deal lines on the kitchen screen (Web Orders) show a "Deal" badge.

Tables

Table	Columns	Purpose
`rs_deals`	id, name, description, price, show_on_website, is_active	The deal and its price.
`rs_deal_items`	id, deal_id, stock_id, attribute_id	The items of each deal. One row per stock item.
`rs_order_items`	+ deal_id	Marks an order line as part of a deal.

On the API

- `GET /deals` returns active deals with `show_on_website`. A deal appears only if every item has a website dish that can be sold now. It also returns the normal price and the saving.
- `POST /orders` accepts `{ deal_id, qty }`. The deal is expanded into one line per item. The deal price is split across the lines in proportion to each item's normal price, and the last line takes any rounding cent, so the lines add up to the deal price exactly.

On the website

- The Hot Deals section lists the deals from `/deals`. Each card shows the items and the saving.
- Deals get their own cart ID range, so a deal never clashes with a dish in the cart.
- The checkout sends a deal as `deal_id`. The server computes the price.

> **Example that was tested:** a deal of Zinger Burger (390) and White Chicken Biryani Medium (350) for 900. The website shows a normal price of 740 and a deal price of 900. The lines came out as 474.32 + 425.68 = 900.00.

7. Menu, sizes and modifiers

Dishes and images

- Every dish in the ERP website menu is shown, grouped by its category. Hidden or sold-out dishes are not sold.
- Image paths saved in the ERP (for example `/demos3/assets/images/...`) are sent to the website as full URLs, so the website can load them.

Sizes from stock attributes

- One stock item can have attributes (for example Single, Medium and Full). The website shows one card for the stock item, with the lowest price as "From".
- The product modal lists each attribute with its own price. The chosen attribute is the dish that is ordered, so stock follows the attribute.

Modifiers

- Modifier groups and options (for example Extras: Cheese +50) are stored in `rs_modifier_groups` and `rs_modifier_options`.

- A group marked as a size replaces the dish price. The other groups add to it. The server checks the required groups and computes the price.

> **Note:** the ERP has no modifier editor screen yet. Modifier groups are currently added in the database.

8. Orders and delivery areas

Web Orders screen

- Filter chips show counts for Active, Received, Preparing, On the way, Completed, Cancelled and All. A second row filters by Delivery or Pickup.
- A search box filters by order code, customer name, phone or area. Auto-refresh stops while searching.
- Every order that is not invoiced can be cancelled. An invoiced order shows "Invoiced" instead, because it has to be reversed with a Sales Return in the ERP.
- Each card shows the customer and address with the delivery area, the items with modifiers and deal badges, the total, and the invoice status.

Delivery areas

- **Restaurant → Delivery Areas:** add an area with its delivery charge and an optional "free over" amount. Leave "free over" empty if that area is never free.
- On the website, delivery orders need an area. The area dropdown shows each area's charge. The server applies the charge, so the browser's figure is never trusted.
- The old global charge (150, free over 3000) is no longer used.

Test	Result
Gulberg (150, free over 3000), Zinger 390	Delivery 150, total 540
DHA (250, never free), Medium 350	Delivery 250, total 600
Gulberg, Wagyu 4800	Free delivery, total 4800

Delivery without an area	Refused: "Please choose your delivery area"

9. Files changed

Restaurant API (`htdocs/restaurant`)

File	What changed
`database/schema.sql`	Tables for modifiers, deals and delivery areas; new columns `show_in_hero` , `deal_id` , `delivery_area` , `replaces_price` .
`api/index.php`	Routes `/menu` (hero, stock grouping, modifiers, image URLs), `/deals` , `/areas` ; `POST /orders` handles modifiers, deals and areas.
`api/config.php`	`public_url` for ERP image paths.

ERP (`demos3`)

File	What changed
`stock/stock_management.php` , `stock/op_save_stock.php`	"Show on hero slider" switch, saved with the item.
`includes/restaurant/cls_RestaurantMenu.php`	Hero flag save.
`includes/restaurant/cls_RestaurantDeal.php` (new)	Deal save with validation, deal lists, the deal view and the report data.
`includes/restaurant/cls_RestaurantDeliveryArea.php` (new)	Delivery area save and list.
`includes/restaurant/cls_RestaurantOrder.php`	Order filters with type, status counts, cancel and modifier display.
`restaurant/deals.php` , `op_save_deal.php` (new)	Deals tab: form, items table and deal list.

`restaurant/deals_report.php` , `deal_view.php` (new)	Deals Report with the eye icon, and the deal detail page.
`restaurant/delivery_areas.php` , `op_save_delivery_area.php` (new)	Delivery Areas tab.
`restaurant/manage_web_orders.php`	New filters, search, order card design and the cancel button.
`restaurant/_tabs.php` , `manage_menu.php`	Tab bar: Web Orders, Website Menu, Deals, Deals Report, Delivery Areas.
`includes/inc_header.php`	Sidebar links: Deals, Deals Report, Delivery Areas.

Website (`htdocs/restnext`)

File	What changed
`lib/data.ts`	Builds the page from `/menu` and `/deals` : hero, categories, popular, deals, variants. Falls back to sample data.
`lib/fetcher.ts`	Server call to the API with the key, and the browser call through the proxy.
`lib/library.ts` , `lib/types.ts`	Cart, checkout with area and deal lines, order tracking with real statuses.
`app/api/[...path]/route.ts` (new)	Proxy that allows only `menu, deals, areas, orders, health` .
`components/CategorySlider.tsx`	Category bar redesigned.
`components/ProductCard.tsx` , `ProductModal.tsx`	"From" price, size picker, sold-out state.
`components/CheckoutForm.tsx` , `OrderTracker.tsx` , `CartDrawer.tsx`	Area dropdown and charge, pickup or delivery, live tracking, larger close button.

`.env.local` (new), `next.config.mjs`	API address and key (server only); image host for ERP photos.

10. Running and checking

Start the services

1. Start Apache and MySQL in XAMPP.
2. Check the API: `http://localhost/restaurant/api/index.php?route=/health`
3. In `htdocs/restnext`, run `npm run build` and then `npm start`, or `npm run dev` while developing.
4. Open `http://localhost:3000`.

What was checked

- API: menu with hero, stock grouping and modifiers; deals with split prices; delivery fees for each area case; refusal of wrong input.
- Website: TypeScript check and production build pass; hero, categories and deals render from the ERP data (screenshots above).
- ERP: PHP syntax checks on every changed file; model tests for save, validation and reports.

> **Not yet checked in a browser:** ERP screens that need a login (Deals form, Delivery Areas, the Web Orders filters and cancel). Please test those once with your login.

11. Open items

- An ERP screen for modifier groups (currently added in the database).
- Cancelling an invoiced order through a Sales Return (currently refused, with a note).
- The Single size for the stock item is a separate attribute in the ERP. It needs stock if stock is tracked for that item.

- The old deal "Burger Sand" uses stock ids that are not stock items (they were menu ids). It is inactive, but it can be deleted.
- Site-wide: keep the production key (`RESTAURANT_API_KEY`) out of the browser. It is set only in `.env.local`.